Compresión y Seguridad: Práctica 1

Memoria de implementación – Primera fase

Pedro González Sánchez
Universidad de Alicante
pgs96@alu.ua.es

Rafael Verdeguer Mora
Universidad de Alicante
rvm57@alu.ua.es

Aitor García Gómez
Universidad de Alicante
agg437@alu.ua.es

Carlos Bravo Jiménez
Universidad de Alicante
cbj13@alu.ua.es

Curso 2026–2027

Resumen

Este trabajo implementa una aplicación de escritorio para cifrar y recuperar archivos mediante un esquema híbrido AES y RSA. Cada archivo se cifra con una clave AES-128 aleatoria en modo GCM; la clave AES se cifra con RSA-OAEP y la clave privada RSA se protege con AES-GCM mediante una clave derivada de una contraseña. La aplicación gráfica permite almacenar y consultar el contenido cifrado y las claves protegidas en una base de datos SQLite local. Esta memoria describe la arquitectura implementada, las decisiones de almacenamiento, el uso de la aplicación, las comprobaciones realizadas y las funcionalidades que quedan pendientes respecto a la segunda fase del enunciado.

Índice

1. Introducción **3**

2. Alcance del trabajo **3**

3. Tecnologías y dependencias **3**

4. Arquitectura **4**

5. Diseño criptográfico **4**

5.1. Cifrado de cada archivo 4

5.2. Protección de la clave AES 4

5.3. Protección de la clave privada RSA 5

6. Proceso de cifrado y descifrado **5**

6.1. Cifrado 5

6.2. Descifrado 5

7. Base de datos SQLite **6**

7.1. Esquema 6

8. Interfaz y manual de usuario **6**

8.1. Preparación 6

8.2. Pestaña «Cifrar» 7

8.3. Pestaña «Biblioteca SQLite» 7

8.4. Pestaña «Descifrar» 7

9. Uso desde la línea de comandos **8**

10. Pruebas y comprobaciones **8**

11. Seguridad, límites y trabajo futuro **8**

11.1. Precauciones 8

11.2. Estado respecto al enunciado 9

12. Entrega **9**

12.1. Repositorio y descarga 9

12.2. Vídeo de demostración 9

1. Introducción

El objetivo de la primera fase es proteger archivos multimedia con una clave simétrica aleatoria distinta para cada archivo y evitar que las claves AES almacenadas queden expuestas. Para ello se combinan algoritmos simétricos y asimétricos:

- AES-128-GCM protege el contenido de cada archivo y permite detectar alteraciones.
- RSA-OAEP protege la clave AES aleatoria asociada a ese archivo.
- AES-128-GCM, con una clave derivada de la contraseña, protege la clave privada RSA.
- SQLite conserva el contenido cifrado y el material criptográfico protegido de la aplicación gráfica.

La aplicación está implementada en Python y cuenta con una interfaz de escritorio en Tkinter. El diseño separa la interfaz, la criptografía, el almacenamiento y los servicios que conectan esas piezas.

2. Alcance del trabajo

La versión actual implementa un flujo local de cifrado y descifrado para archivos. Trabaja sobre bytes, por lo que puede cifrar imágenes, audio, vídeo y otros tipos de archivo; no se limita a una extensión concreta.

La aplicación gráfica permite crear cuentas locales y autenticar al usuario mediante la contraseña que desbloquea su clave privada RSA. Cada cuenta posee su propio par RSA y sus propios archivos. La compartición segura entre usuarios todavía no está implementada.

3. Tecnologías y dependencias

- **Python 3.9 o posterior**: lenguaje de implementación.
- **PyCryptodome**: implementación de AES, RSA y generación de bytes aleatorios criptográficamente seguros. La dependencia del proyecto está declarada en `requirements.txt`.
- **Tkinter**: interfaz gráfica, mediante la biblioteca estándar de Python.
- **SQLite**: base de datos local, mediante el módulo `sqlite3` de la biblioteca estándar; no requiere un servidor independiente.
- **unittest**: pruebas automatizadas de la capa de almacenamiento y del flujo criptográfico.

La interfaz gráfica también requiere que Tkinter esté instalado en el intérprete de Python. Algunas distribuciones de Linux ofrecen Tkinter como paquete del sistema separado.

4. Arquitectura

El proyecto se organiza en módulos para evitar mezclar la interfaz y las consultas SQL con las operaciones criptográficas:

`crypto/aes.py`
Genera claves AES-128 y proporciona funciones para cifrar y descifrar bytes o archivos con AES-GCM.

`crypto/rsa.py`
Genera un par RSA de 2048 bits y cifra o descifra claves simétricas con RSA-OAEP.

`crypto/passwords.py`
Deriva claves desde contraseñas con PBKDF2-HMAC-SHA256 y protege o recupera la clave privada RSA.

`database/database.py`
Inicializa el esquema SQLite y proporciona operaciones de lectura y escritura parametrizadas.

`services/file_service.py`
Coordina cifrado, RSA y persistencia en la base de datos, sin depender de los widgets de Tkinter.

`main.py`
Crea la interfaz gráfica, valida la selección de archivos y solicita al servicio las operaciones.

`encrypt.py` y `decrypt.py`
Conservan un flujo alternativo de línea de comandos que guarda el contenido cifrado como archivos externos.

5. Diseño criptográfico

5.1. Cifrado de cada archivo

Para cada archivo nuevo, la aplicación genera una clave aleatoria de 16 bytes:

$$16 \text{ bytes} \times 8 = 128 \text{ bits}.$$

Esa clave se utiliza una sola vez para cifrar el contenido con AES en modo Galois/Counter (GCM). GCM produce el texto cifrado y una etiqueta de autenticación. La implementación guarda juntos el nonce, la etiqueta y el texto cifrado. Al descifrar, la etiqueta se verifica antes de escribir el contenido recuperado; si la autenticación falla, la operación produce un error.

5.2. Protección de la clave AES

RSA se utiliza para cifrar la clave AES, no el archivo completo. La clave pública cifra los 16 bytes de la clave AES con RSA-OAEP. Durante la recuperación, la clave privada

correspondiente descifra ese material. Esta combinación permite usar el cifrado simétrico para archivos y el asimétrico para transportar la clave de cada archivo.

5.3. Protección de la clave privada RSA

La contraseña de la cuenta se utiliza exclusivamente para proteger y desbloquear su clave privada RSA; no cifra directamente los archivos ni sus claves AES. La contraseña no se almacena. Para proteger la clave privada RSA:

1. Se genera un salt aleatorio de 16 bytes.
2. PBKDF2-HMAC-SHA256 deriva 16 bytes de clave, con 200 000 iteraciones en la implementación actual.
3. La clave derivada cifra la representación DER de la clave privada con AES-128-GCM.
4. Se conserva el salt junto con el nonce, la etiqueta de autenticación y el texto cifrado de la clave privada.

El salt no es secreto. Permite repetir la derivación durante el descifrado. Una contraseña equivocada deriva otra clave y la verificación de GCM falla. Al iniciar sesión, la clave privada se desbloquea una vez y se conserva en memoria durante esa sesión; la interfaz no vuelve a solicitar la contraseña para cada operación.

6. Proceso de cifrado y descifrado

6.1. Cifrado

Desde la interfaz gráfica, el proceso es el siguiente:

1. El usuario inicia sesión con su cuenta y desbloquea su clave privada RSA una vez.
2. Elige un archivo que se cifra con una clave AES aleatoria nueva y AES-GCM.
3. La clave AES se cifra con RSA-OAEP y la clave pública de la cuenta activa.
4. El texto cifrado y la clave AES cifrada se guardan en SQLite junto con metadatos del archivo.

6.2. Descifrado

Para recuperar un registro desde la biblioteca SQLite:

1. Se selecciona un registro de la cuenta autenticada. Su clave privada ya está desbloqueada en memoria.
2. RSA descifra la clave AES asociada al registro.
3. AES-GCM descifra y autentica el contenido.

4. Solo tras superar la autenticación, se escribe el archivo en la ubicación elegida.

La aplicación también mantiene una pestaña para descifrar archivos externos `.enc` y `.key.enc`. La clave AES debe haberse cifrado para la clave pública de la cuenta activa. Esos archivos no se importan automáticamente a la biblioteca SQLite.

7. Base de datos SQLite

La interfaz crea automáticamente la base de datos local `secure_storage.sqlite3`. Durante el desarrollo se ubica en `data/`. Cuando se ejecuta como aplicación empaquetada, el código está preparado para usar la carpeta del usuario `%LOCALAPPDATA%\CS2026`, para no intentar escribir junto al ejecutable.

7.1. Esquema

Tabla	Campos principales	Contenido y propósito
`users`	`id`, `username`, `public_key`, `encrypted_private_key`, `created_at`	Almacena una cuenta y su par RSA. La clave privada está cifrada con AES-GCM; la contraseña no se guarda.
`encrypted_files`	`id`, `owner_id`, `original_name`, `encrypted_content`, `encrypted_aes_key`, `created_at`, `size_bytes`	Almacena el propietario, el contenido cifrado y la clave AES cifrada como BLOB, además de metadatos.

Las relaciones se protegen con claves foráneas de SQLite. Las consultas que insertan datos usan parámetros SQL, y las escrituras se realizan en transacciones. La biblioteca filtra por el identificador del usuario autenticado.

El contenido original y la clave AES en claro no se guardan en la base de datos. Sin embargo, SQLite no cifra automáticamente el fichero completo: los nombres, tamaños y fechas se conservan como metadatos legibles, y el archivo de base de datos debe mantenerse en un lugar protegido. Los archivos se leen y cifran íntegramente en memoria, por lo que esta primera versión está orientada a archivos de tamaño moderado.

Al abrir una base de datos de la versión anterior, el antiguo par RSA y sus archivos se migran a una cuenta local llamada `usuario`. Para iniciar sesión en esa cuenta se necesita la contraseña que protegía el par anterior.

8. Interfaz y manual de usuario

8.1. Preparación

Desde la carpeta raíz del proyecto, se crea y activa un entorno virtual.

Linux o macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Con el entorno activado, se instala PyCryptodome y se inicia la interfaz:

```
python -m pip install -r requirements.txt
python main.py
```

8.2. Pestaña «Cifrar»

Al abrir la aplicación, se elige una cuenta y se introduce su contraseña. Si todavía no hay cuentas, se puede crear una; durante el alta se genera un par RSA propio y se protege la clave privada con la contraseña elegida.

1. Seleccionar el archivo original con el botón de exploración.
2. Pulsar **Cifrar y guardar en SQLite**.
3. La aplicación cifra para la cuenta activa e informa del identificador del registro guardado.

El contenido cifrado y las claves se guardan en la base de datos local. No se genera un archivo cifrado externo en este flujo gráfico.

8.3. Pestaña «Biblioteca SQLite»

La biblioteca muestra el identificador, nombre original, fecha de almacenamiento y tamaño de los registros de la cuenta activa. Se puede actualizar la lista y elegir un registro para descifrarlo. La clave privada desbloqueada al iniciar sesión recupera el archivo; solo hay que seleccionar el destino. La interfaz confirma antes de sobrescribir un destino que ya exista.

8.4. Pestaña «Descifrar»

Esta pestaña sirve para archivos externos. Se eligen el archivo cifrado, su clave AES cifrada `.key.enc` y la ruta de salida. Se utiliza la clave privada de la cuenta activa y no se vuelve a pedir la contraseña. El modo de línea de comandos es independiente y recibe `-password` en cada ejecución; solo sus archivos cifrados con la clave pública de la cuenta activa se pueden abrir desde esta pestaña.

9. Uso desde la línea de comandos

El modo de consola es compatible con el flujo de ficheros externos.

Cifrar:

```
python encrypt.py /ruta/al/archivo.jpg --password "mi-contraseña"
```

Por defecto, los archivos cifrados y las claves se crean en `data/`. Se puede cambiar la ubicación de salida:

```
python encrypt.py /ruta/al/archivo.jpg \
  --password "mi-contraseña" \
  --output-dir /ruta/de/salida
```

Descifrar:

```
python decrypt.py \
  --encrypted-file data/archivo.enc \
  --key-file data/archivo.key.enc \
  --password "mi-contraseña" \
  --output /ruta/recuperada/archivo.jpg
```

Si la clave privada protegida no está junto al archivo cifrado ni en `data/`, se indica su ruta mediante la opción `-private-key`.

10. Pruebas y comprobaciones

La suite `tests/test_file_service.py` contiene pruebas automatizadas para:

- Cifrar un archivo, persistir el contenido y recuperar bytes idénticos al original.
- Crear cuentas, desbloquear sus claves privadas y rechazar contraseñas incorrectas.
- Aislar los archivos de cada usuario y evitar que otra cuenta los descifre.
- Migrar el esquema anterior de un solo par RSA conservando los archivos cifrados.
- Rechazar nombres de usuario duplicados.

Además, se ha comprobado manualmente la inicialización de la ventana Tkinter con la pestaña de biblioteca SQLite y se ha validado que el flujo de línea de comandos de cifrado/descifrado sigue siendo compatible con ficheros externos.

11. Seguridad, límites y trabajo futuro

11.1. Precauciones

- Deben conservarse la base de datos SQLite y la contraseña para recuperar archivos guardados desde la interfaz.

- Perder la contraseña o la clave privada protegida hace irrecuperables los archivos asociados.
- La base de datos contiene datos cifrados, pero no es una base de datos cifrada íntegramente. Se recomienda proteger el acceso al dispositivo y realizar copias de seguridad privadas.
- El nombre original, el tamaño y la fecha de almacenamiento no se cifran.
- `.gitignore` excluye datos locales y claves, pero no elimina archivos que Git ya siga ni información publicada anteriormente. Antes de compartir el repositorio debe revisarse su historial y su estado.

11.2. Estado respecto al enunciado

La primera fase está implementada a nivel local: cifrado AES-128 por archivo, envoltura RSA de la clave AES, protección de la clave privada, cuentas locales con inicio de sesión y persistencia SQLite en la interfaz. La aplicación también ofrece un flujo por consola independiente.

Queda pendiente la segunda fase del enunciado:

1. Compartición segura, cifrando la clave AES del archivo con la clave pública del destinatario y registrando el permiso correspondiente.

12. Entrega

12.1. Repositorio y descarga

El código fuente del proyecto está disponible públicamente en GitHub, donde se pueden consultar los archivos de la aplicación y su desarrollo.

- **Repositorio de GitHub:** El-magoXD/CS2026.
- **Descarga del ejecutable para Windows:** CS2026.exe (versión 1.0).

El ejecutable permite utilizar la aplicación sin necesidad de ejecutar manualmente el código fuente de Python.

12.2. Vídeo de demostración

A continuación, se proporciona el enlace al vídeo de demostración del proyecto, en el que se muestran las funcionalidades implementadas hasta el momento.

- **Vídeo demo:** Ver demostración en YouTube.